

Practical - 6

Practical Title: Rendering External JSON Data using Fetch API, Search and Filter

Problem Definition: Display event lists, student profiles, notices, or FAQs from external JSON files using Fetch API. Implement dynamic rendering, search, filter, sorting, and pagination.

A. Theory

1. How is JSON fetched, parsed, and rendered?

Answer:

JSON data can be stored in an external .json file such as events.json.

JavaScript uses the Fetch API to request the JSON file:

```
fetch("events.json")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  });
```

The process has three main steps:

Step 1: Fetch the JSON

```
fetch("events.json")
```

fetch() sends a request to get the JSON file. The Fetch API returns a Promise that eventually provides a Response object.

Step 2: Parse the JSON

```
response.json()
```

The json() method reads the response and converts the JSON data into a JavaScript value, such as an object or array.

Step 3: Render the data

After receiving the data, JavaScript can use methods such as map() to create HTML elements and display the information on the webpage.

Example:

```
data.map(event => {
  return `<div>${event.title}</div>`;
});
```

The generated HTML can then be inserted into the webpage.

Reference Link:

https://www.w3schools.com/js/js_json_server.asp

https://www.w3schools.com/Js/js_api_fetch.asp

2 Which array methods are used for search, filter, sort, and pagination?

Answer:

JavaScript provides several array methods that are useful when working with JSON data.

map()

map() creates a new array by performing an operation on every element.

For example:

```
let names = students.map(student => student.name);
```

It can be used to convert JSON records into HTML cards or table rows.

filter()

filter() creates a new array containing only the elements that satisfy a condition.

Example:

```
let result = students.filter(student => student.course === "WDF");
```

This can be used to show only students from a particular course.

For a search:

```
let result = students.filter(student =>
  student.name.toLowerCase().includes(searchText.toLowerCase())
);
```

W3Schools describes filter() as creating a new array containing elements that pass a test.

sort()

sort() is used to arrange array elements.

For example:

```
students.sort((a, b) => a.name.localeCompare(b.name));
```

This can arrange students alphabetically.

W3Schools provides sort() for sorting arrays and also covers sorting objects.

Pagination

Pagination means dividing a large list into smaller pages.

For example, if there are 15 records and we want to display 5 records per page:

Page 1 → Records 1–5

Page 2 → Records 6–10

Page 3 → Records 11–15

JavaScript can use:

```
let start = (currentPage - 1) * recordsPerPage;
```

```
let end = start + recordsPerPage;
```

```
let pageData = data.slice(start, end);
```

Then only page Data is rendered

Reference Link:

https://www.w3schools.com/js/js_array_iteration.asp

https://www.w3schools.com/js/js_array_sort.asp

3. How are loading and error states handled?

Answer:

Fetching external data takes some time. Therefore, the website should show a loading message while the data is being retrieved.

Example:

```
async function loadData() {  
  try {  
    document.getElementById("status").textContent = "Loading...";  
  
    const response = await fetch("events.json");  
  
    if (!response.ok) {  
      throw new Error("Failed to load data");  
    }  
  
    const data = await response.json();  
  
    document.getElementById("status").textContent = "";  
    displayEvents(data);  
  
  } catch (error) {  
    document.getElementById("status").textContent =  
      "Unable to load events.";  
  }  
}
```

There are three possible states:

1. Loading

Loading events...

2. Success

Events loaded successfully.

3. Error

Unable to load events.

An important point is that `fetch()` does not automatically reject for HTTP errors such as 404, so `response.ok` should be checked. W3Schools specifically recommends checking `response.ok` and using `try...catch` for asynchronous errors

Reference Link:

https://www.w3schools.com/Js/js_api_fetch.asp

https://www.w3schools.com/js/js_async_await.asp

4. How is modularity maintained in JavaScript files?

Answer:

Modularity means **dividing a large JavaScript program into smaller files**, where each file performs a specific task.

For example:

```
project/
|
|— index.html
|— style.css
|
|— data.js
|— render.js
|— search.js
|— pagination.js
|— main.js
```

Each file can have a separate responsibility.

data.js

Responsible for fetching JSON:

```
export async function loadData() {
  const response = await fetch("events.json");
  return await response.json();
}
```

render.js

Responsible for displaying data:

```
export function renderEvents(events) {
  // Generate and display HTML
}
```

main.js

Imports and uses these functions:

```
import { loadData } from "./data.js";
```

```
import { renderEvents } from "./render.js";
```

This makes the program easier to understand, maintain, debug, and reuse. W3Schools explains that JavaScript modules use export and import to divide code into separate files

Reference Link:

https://www.w3schools.com/js/js_modules.asp

B. Implementation/ Output/Result

events.json file

```
C. [
D.   { "id": 1, "title": "Web Development Workshop", "description":
      "Hands-on session covering HTML, CSS and JavaScript basics.",
      "category": "Workshop", "date": "2026-10-05" },
E.   { "id": 2, "title": "Hackathon 2026", "description": "24-hour coding
      challenge – form your team and register now.", "category":
      "Competition", "date": "2026-10-12" },
F.   { "id": 3, "title": "Technical Seminar", "description": "Guest
      lecture on emerging trends in software development.", "category":
      "Seminar", "date": "2026-10-18" },
G.   { "id": 4, "title": "C++ Bootcamp", "description": "Master object-
      oriented programming concepts and memory management.", "category":
      "Workshop", "date": "2026-10-22" },
H.   { "id": 5, "title": "DSA Mock Test", "description": "Prepare for
      technical interviews with dynamic problem solving.", "category":
      "Exam", "date": "2026-11-01" },
I.   { "id": 6, "title": "AI & ML Symposium", "description": "Explore
      recent breakthroughs in machine learning models.", "category":
      "Seminar", "date": "2026-11-08" },
J.   { "id": 7, "title": "Open Source Sprint", "description": "Contribute
      to popular open-source projects with expert guidance.", "category":
      "Workshop", "date": "2026-11-15" },
K.   { "id": 8, "title": "Cybersecurity Webinar", "description": "Learn
      network security fundamentals and ethical hacking basics.", "category":
      "Seminar", "date": "2026-11-20" },
L.   { "id": 9, "title": "Cloud Computing Summit", "description": "AWS and
      Azure architecture deep-dive for beginner developers.", "category":
      "Seminar", "date": "2026-11-28" },
M.   { "id": 10, "title": "UI/UX Design Jam", "description": "Create
      intuitive web mockups in a collaborative design sprint.", "category":
      "Competition", "date": "2026-12-03" },
```

```

N.  { "id": 11, "title": "Database Optimization Tech Talk",
      "description": "Learn indexing, querying, and schema design
strategies.", "category": "Seminar", "date": "2026-12-10" },
O.  { "id": 12, "title": "Mobile App Dev Workshop", "description": "Build
cross-platform mobile apps using modern frameworks.", "category":
"Workshop", "date": "2026-12-15" },
P.  { "id": 13, "title": "Annual Code Meetup", "description": "Networking
event for students, alumni, and industry mentors.", "category":
"Networking", "date": "2026-12-20" },
Q.  { "id": 14, "title": "DevOps Practices Demo", "description":
"Automate CI/CD pipelines and software deployment.", "category":
"Workshop", "date": "2026-12-24" },
R.  { "id": 15, "title": "Project Showcase Day", "description": "Final
semester project presentations and awards.", "category": "Exhibition",
"date": "2026-12-30" }
S. ]

```

faqs.json file

```

[
  { "id": 1, "question": "How do I check my attendance?", "answer": "Navigate to the
Attendance tab in the navigation bar to view your percentage.", "category": "Academic"
},
  { "id": 2, "question": "Where can I view my assignment due dates?", "answer": "All
pending and submitted assignments are listed on the Assignment page.", "category":
"Academic" },
  { "id": 3, "question": "How do I toggle Dark Mode?", "answer": "Click the moon/sun
icon located at the top right of the navigation bar.", "category": "General" },
  { "id": 4, "question": "Who can register on StudentHub?", "answer": "Any enrolled
student can register using the Sign Up page.", "category": "General" },
  { "id": 5, "question": "What is the minimum attendance required?", "answer": "A
minimum of 75% attendance is required to maintain 'Good' status.", "category":
"Academic" },
  { "id": 6, "question": "How are grades calculated?", "answer": "Marks above 90 earn
A+, 80-89 earn A, 70-79 earn B, and below 70 earn C.", "category": "Academic" },
  { "id": 7, "question": "How can I contact my course faculty?", "answer": "Faculty
details and materials are listed on individual Course pages.", "category": "Support" },

```

```
{ "id": 8, "question": "What should I do if I forget my password?", "answer": "Please
contact the system administrator or visit the Settings page.", "category": "Support" },

{ "id": 9, "question": "Is the StudentHub mobile friendly?", "answer": "Yes, click the
hamburger menu on smaller screens to navigate.", "category": "General" },

{ "id": 10, "question": "Where can I download course slides?", "answer": "Course
materials are hyperlinked on each subject detail page.", "category": "Academic" },

{ "id": 11, "question": "How often is the attendance updated?", "answer": "Attendance
percentages update weekly on the portal.", "category": "Academic" },

{ "id": 12, "question": "Can I edit my profile details?", "answer": "You can view and
manage your profile details on the Settings page.", "category": "Support" },

{ "id": 13, "question": "How do I register for Hackathons?", "answer": "Check the
Upcoming Events section on the Home page for event registration.", "category":
"Events" },

{ "id": 14, "question": "Who is the faculty for C++?", "answer": "Premal Sir teaches the
Programming with C++ course.", "category": "Academic" },

{ "id": 15, "question": "Who teaches Data Structure and Algorithms?", "answer":
"Pravin Sir teaches the Data Structure and Algorithm course.", "category": "Academic" }
]
```

students.json file

```
[

{ "id": "ST1001", "name": "Alice Johnson", "email": "alice@gmail.com", "phone":
"9876543210", "course": "WDF", "year": "1" },

{ "id": "ST1002", "name": "Bob Smith", "email": "bob@gmail.com", "phone":
"9876543211", "course": "C++", "year": "2" },

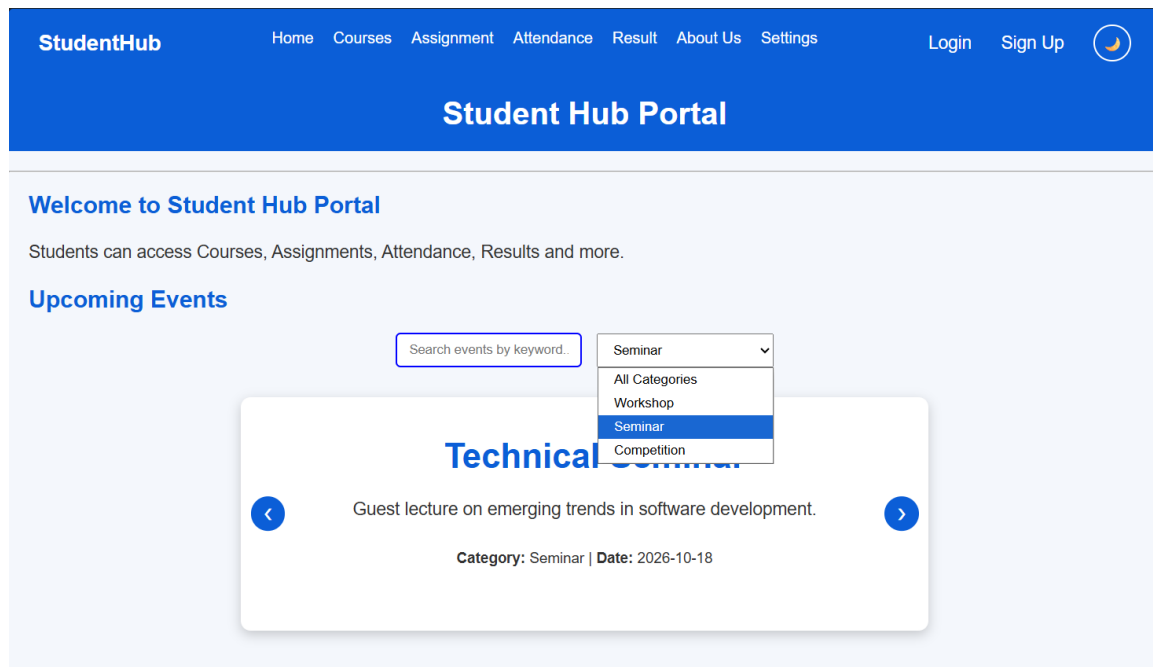
{ "id": "ST1003", "name": "Charlie Brown", "email": "charlie@gmail.com", "phone":
"9876543212", "course": "DSA", "year": "3" },

{ "id": "ST1004", "name": "David Miller", "email": "david@gmail.com", "phone":
"9876543213", "course": "WDF", "year": "4" },

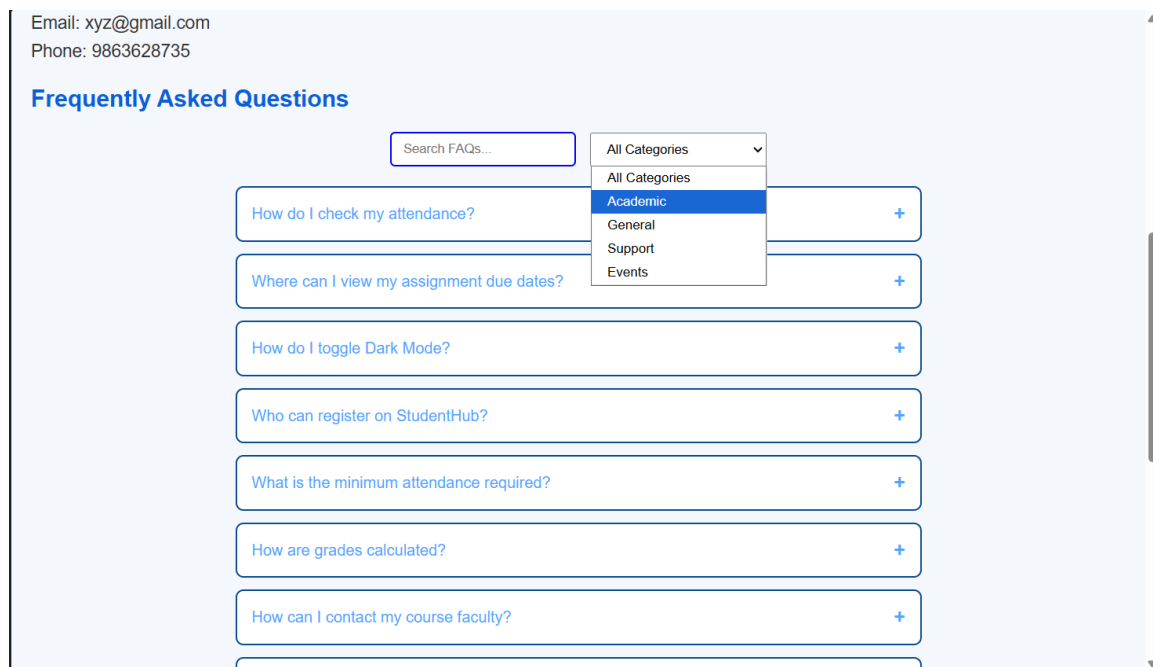
{ "id": "ST1005", "name": "Emma Wilson", "email": "emma@gmail.com", "phone":
"9876543214", "course": "C++", "year": "1" },
```

```
{ "id": "ST1006", "name": "Frank Thomas", "email": "frank@gmail.com", "phone":  
"9876543215", "course": "DSA", "year": "2" },  
  
{ "id": "ST1007", "name": "Grace Lee", "email": "grace@gmail.com", "phone":  
"9876543216", "course": "WDF", "year": "3" },  
  
{ "id": "ST1008", "name": "Hannah Davis", "email": "hannah@gmail.com", "phone":  
"9876543217", "course": "C++", "year": "4" },  
  
{ "id": "ST1009", "name": "Ian Clark", "email": "ian@gmail.com", "phone":  
"9876543218", "course": "DSA", "year": "1" },  
  
{ "id": "ST1010", "name": "Jack Martinez", "email": "jack@gmail.com", "phone":  
"9876543219", "course": "WDF", "year": "2" },  
  
{ "id": "ST1011", "name": "Karen Taylor", "email": "karen@gmail.com", "phone":  
"9876543220", "course": "C++", "year": "3" },  
  
{ "id": "ST1012", "name": "Liam Anderson", "email": "liam@gmail.com", "phone":  
"9876543221", "course": "DSA", "year": "4" },  
  
{ "id": "ST1013", "name": "Mia White", "email": "mia@gmail.com", "phone":  
"9876543222", "course": "WDF", "year": "1" },  
  
{ "id": "ST1014", "name": "Noah Harris", "email": "noah@gmail.com", "phone":  
"9876543223", "course": "C++", "year": "2" },  
  
{ "id": "ST1015", "name": "Olivia Martin", "email": "olivia@gmail.com", "phone":  
"9876543224", "course": "DSA", "year": "3" }  
]
```

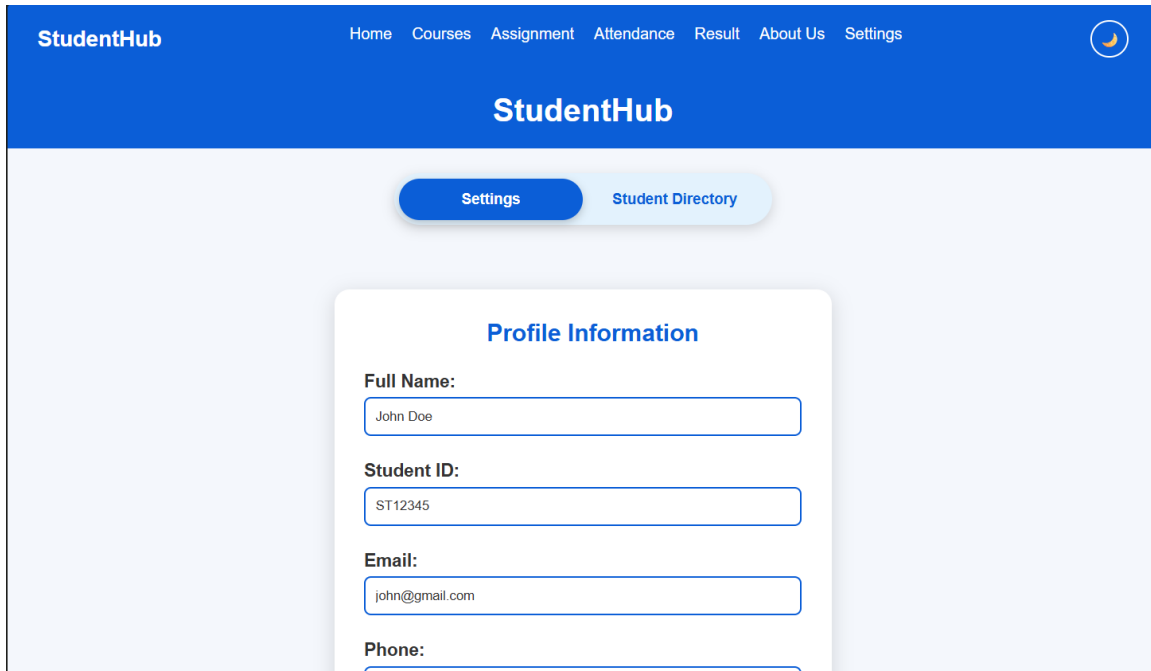
T. Evidence of Work (Screenshots)



Description: The screenshot shows the Upcoming Events section of StudentHub. Event data is dynamically loaded from events.json using the Fetch API. The search box and category dropdown allow users to search and filter events, while the event card displays the event title, description, category, and date.



Description: The screenshot shows the FAQ section of StudentHub. FAQs are loaded dynamically from faqs.json and displayed on the webpage. Users can search for questions and filter them by categories such as Academic, General, Support, and Events. The + button can be used to expand an FAQ and view its answer.



The screenshot displays the StudentHub interface. At the top, a blue navigation bar contains the 'StudentHub' logo on the left and a series of links: Home, Courses, Assignment, Attendance, Result, About Us, and Settings. On the far right of this bar is a circular profile icon. Below the navigation bar, a large blue banner also features the 'StudentHub' logo. Underneath the banner, there are two buttons: 'Settings' (highlighted in dark blue) and 'Student Directory' (light blue). The main content area is a light blue gradient. Centered in this area is a white card titled 'Profile Information'. This card contains four form fields, each with a label and a text input: 'Full Name:' with the value 'John Doe', 'Student ID:' with the value 'ST12345', 'Email:' with the value 'john@gmail.com', and 'Phone:' which is currently empty.

Description: The screenshot shows the Settings page of StudentHub, containing profile information such as full name, student ID, email, and phone number. The user can update the information using the Save Changes button. The page also provides a logout option and navigation to the Student Directory.

Profile Information

Full Name:

Student ID:

Email:

Phone:

[Save Changes](#)

[Logout](#)

Description: The screenshot shows the Student Directory, where student records are dynamically loaded from students.json. The records are displayed in a table containing ID, name, email, course, and year. A search box and course filter are provided to help users find specific student records.

StudentHub Home Courses Assignment Attendance Result About Us Settings

StudentHub

[Settings](#) [Student Directory](#)

Student Directory

View student records loaded dynamically from JSON.

Search by name or ID... All Courses Name (A-Z)

Course Filter: All Courses, WDF, C++, DSA

ID	Name		Course	Year
ST1001	Alice Johnson	alice@gmail.com	WDF	Year 1
ST1002	Bob Smith	bob@gmail.com	C++	Year 2
ST1003	Charlie Brown	charlie@gmail.com	DSA	Year 3

Description: The screenshot demonstrates the course filtering feature in the Student Directory. The dropdown provides options such as WDF, C++, and DSA. When a course is selected, JavaScript filters the JSON data and displays only the students belonging to that selected course.

Student Directory

View student records loaded dynamically from JSON.

All Courses

Name (A-Z)

Name (A-Z)

Name (Z-A)

ID (Ascending)

ID	Name	Email	Course	Year
ST1001	Alice Johnson	alice@gmail.com	WDF	Year 1
ST1002	Bob Smith	bob@gmail.com	C++	Year 2
ST1003	Charlie Brown	charlie@gmail.com	DSA	Year 3
ST1004	David Miller	david@gmail.com	WDF	Year 4
ST1005	Emma Wilson	emma@gmail.com	C++	Year 1

1

2

3

Description: The screenshot demonstrates sorting and pagination in the Student Directory. Users can sort students by name in ascending or descending order or by ID. The numbered buttons at the bottom allow users to navigate between different pages of student records.

U. GitHub Repository Link:

<https://github.com/devanshirshah24-pixel/WDF-Practicals/tree/main/Practical6>

V. Challenges Faced:

1. Fetching the JSON file

Understanding how `fetch()` works and how to correctly specify the JSON file path was initially challenging.

2. Parsing JSON data

Understanding the difference between JSON data and JavaScript objects/arrays was a challenge. The `response.json()` method was used to convert the fetched JSON into JavaScript data.

3. Dynamic rendering

Creating HTML elements dynamically from JSON records required understanding JavaScript arrays, `map()`, DOM manipulation, and template literals.

4. Search and filtering

Implementing search and filtering required understanding how `filter()` works and how user input can be compared with JSON data.

5. Sorting data

Sorting objects based on properties such as name, date, or category required understanding the `sort()` method and comparison functions.

6. Handling errors

Handling problems such as an incorrect file path, missing JSON file, or unsuccessful HTTP response required proper use of `response.ok try...catch`, and error messages.

7. Pagination

Dividing a large JSON dataset into smaller groups and displaying only the required records required understanding array indexing and `slice()`.

8. Maintaining modular code

Separating fetching, rendering, searching, filtering, and pagination into different JavaScript modules required understanding import and export

W.Learning Outcomes:

After completing this practical, I learned:

1. How to create and structure JSON data using objects and arrays.
2. How to fetch external JSON files using the JavaScript Fetch API.
3. How to parse JSON data using `response.json()`.
4. How to render JSON data dynamically on an HTML webpage.
5. How to use `map()` for processing and displaying array data.

6. How to use filter() to implement search and filtering.
7. How to use sort() to arrange JSON records.
8. How to implement pagination for displaying large datasets.
9. How to handle loading and error states while fetching data.
10. How to divide JavaScript code into modules using import and export.
11. How external JSON data can be used in modern web applications to create dynamic and reusable interfaces.

Faculty Remarks: (Keep this blank for faculty)